

README

Project : Explainable Financial Research Assistant

Product:

Q&A based 10-K report analyzer for 28 Nasdaq companies.

Financial reports are not easy to understand and even harder to use as it can be challenging to vet out required information from the large amount of data. This project builds a Q&A assistant that answers the questions on the topics using the financial data from SEC filings with evidence cited and confidence score.

For example, the assistant will answer questions like this:

Question

What are Apple's biggest risks?

Answer:

Apple faces supply chain concentration and regulatory risks.

Evidence:

AAPL 10-K 2025

Item 1A Risk Factors

Paragraph 2345

Confidence:

93%

Specifically, this project used the data from 28 Nasdaq company's 10K reports to answer the questions related to 10K report of those company for year 2025. Other limitation is on the sections for which this product works. The sections from the 10K report that are used in this product are:

Item 1 - Business

Item 1A - Risk Factors

Item 7 - Management Discussion and Analysis of Financial Conditions

Item 8 - Financial Statements

Working URL:

You can find the product in action here:

<https://financial-assistant-global-22115228676.us-central1.run.app>

Bill Of Material:

The project consists of 5 main components:

1. DataCreation

Ipynb file for creating Q&A pair data by calling sec-io and vertexAI apis

2. FAISS

Ipynb file for creating vector index using FAISS

Data

JSON file with Q&A and meta data for creating index

3. ModelTraining

Ipynb file for training and evaluating the finBERT model

Data

JSON file with positive and negative Q&A and meta data for training.

4. Evaluation

Ipynb file for evaluating FAISS + finBert combination

Data

Data to test and evaluate the FAISS and finBert combination. Returns Recall at 80 and 100 and MRR

5. Service (UI, AnswerRetriever and ReRanker) - monolith

Pycharm project with UI that uses the saved FAISS index and finBERT model to answer the questions

Directory - [CapstoneProject1](#)

Files - Dockerfile (dockerfile to create the docker image)

[main.py](#) (streamlit UI code and module that calls retriever and re-ranker)

requirements.txt (Required python packages for the project)

[retriever.py](#) - (module that feeds top 50 relevant references to the question to re-ranker)

[reranker.py](#) (module that re-ranks the response from retriever and creates response from feeding top 5 references into Vertex AI)

Deployment and build/deploy commands:

The models and FAISS index is stored in gcp bucket and the service docker image is deployed on gcp cloudbuild.

Create and put the saved model and indexes to GCP bucket

Command to store the trained model for re-ranking in gcp bucket:

```
gcloud storage cp config.json gs://ai-capstone-model-retriever/models/finbert_relevance_mv
```

```
gcloud storage cp model.safetensors
```

```
gs://ai-capstone-model-retriever/models/finbert_relevance_mv
```

```
gcloud storage cp tokenizer_config.json
```

```
gs://ai-capstone-model-retriever/models/finbert_relevance_mv
```

```
gcloud storage cp tokenizer.json gs://ai-capstone-model-retriever/models/finbert_relevance_mv
```

Command to store the FAISS index for retrieving in gcp bucket:

```
gcloud storage cp financial_reports.index gs://ai-capstone-model-retriever/faiss
```

```
gcloud storage cp financial_reports_metadata.pkl gs://ai-capstone-model-retriever/faiss
```

Commands to create and tag docker image

```
docker build -t <financial-assistant:sec> .
```

```
docker tag <financial-assistant:sec> <mevashi/financial-assistant:sec>
```

Commands to load the artifacts and deploy to GCP

```
gcloud builds submit --tag
```

```
<us-central1-docker.pkg.dev/triple-mountain-483601-k3/financial-ai/financial-assistant:sec>
```

```
gcloud run deploy <financial-assistant>
--image=us-central1-docker.pkg.dev/triple-mountain-483601-k3/financial-ai/financial-assistant:se
c --platform=managed --region=us-central1 --port=8080 --cpu=4 --memory=8Gi --timeout=300
--min-instances=0 --max-instances=3 --allow-unauthenticated
--set-env-vars=GOOGLE_CLOUD_PROJECT=triple-mountain-483601-k3,GOOGLE_CLOUD_L
OCATION=us-central1
--service-account=triple-mountain-483601-k3@appspot.gserviceaccount.com
```

Scaling number of instances in cloudrun

Search (/) for resources, docs, products, and more

Edit & deploy new revision Connect to repo

Region: us-central1 URL: <https://financial-assistant-globa>

Revisions Source Triggers **Networking** S

from your project, shared VPC, and VPC service controls perimeter. Traffic
er Cloud Run service must be routed through a VPC. Limitations apply.
[Learn more](#)

traffic from external Application Load Balancers

access to your service from the internet

From the latest revision: financial-assistant-globa...

Edit scaling

☒ Auto scaling

Minimum number of instances Maximum number of instances

Set to 1 to reduce cold starts. [Learn more](#)

☐ Manual scaling

Save Cancel

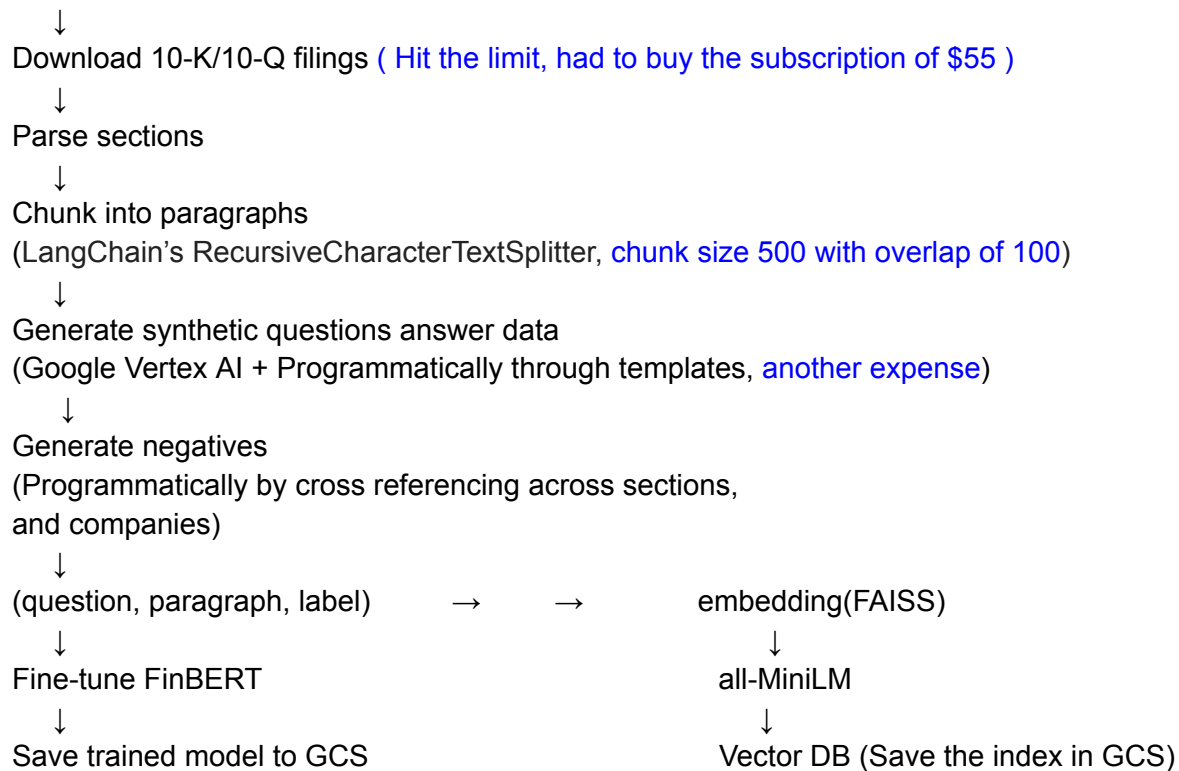
Design

- Download the 10K filings of the companies from SEC using sec-io APIs.
- Chunk the sections using LangChain's RecursiveCharacterTextSplitter
- Use Gemini Vertex AI APIs to create Q&A pairs from the chunk for training (~100K Q&A pairs)
- Generate negative Q&A and metadata pairs programmatically

- Use the Q&A data to create FAISS index and train the FinBert model
- Save the model and index to gcp bucket
- Streamlit for UI to ask questions
- Retriever module grabs 50 most relevant chunks using FAISS index
- Re-ranker filters 5 top relevant chunks using trained FinBert model
- Feed top 5 relevant paragraphs from re-ranker to generate answer using Gemini Vertex AI

OFFLINE TRAINING PHASE

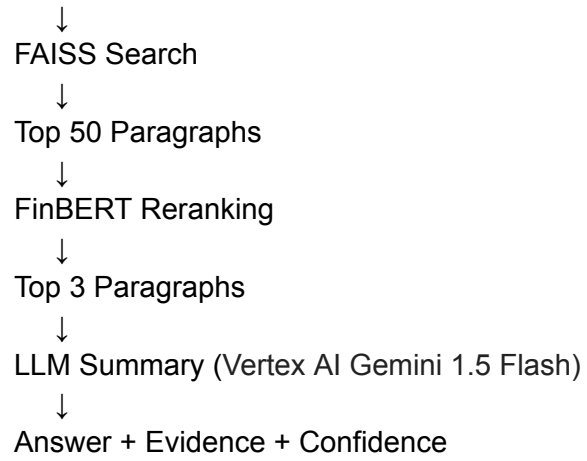
SEC EDGAR



ONLINE INFERENCE PHASE

User Question

↓
all-MiniLM Question Embedding



TODO:

1. CI/CD